

Automatic Creation of Indices in PostgreSQL

Project Submission Report

Name	Roll number
Siddhant Mulkikar	23B0956
Yuvraj Gupta	23B0999
Sandeep Reddy Nallamilli	23B1006
Yash Sabale	23B1043

Public git repository (forked from postgres/postgres):

<https://github.com/HypedUpb0ii/postgres>

The repository is a fork of the upstream PostgreSQL source tree. The initial pull from upstream is preserved as a single ancestor commit; all our work is on top of it under `contrib/auto_index/`, `scripts/`, and `report/`. Per-file diffs against upstream are included in the submission zip under `diffs/`.

1 Motivation

Indexes are the single biggest lever a relational database has on read latency — turning an $O(N)$ filter into an $O(\log N)$ lookup — but they are notoriously hard to manage manually. Workloads change over time, indexes have ongoing write-amplification and storage cost, composite-key ordering is non-obvious ((a, b) or (b, a) ?), and most teams add indexes reactively and never remove them. Oracle 19c ships an “Auto Indexing” feature; SQL Server has Auto Tuning. PostgreSQL has `hypopg` (hypothetical indexes) and `pg_qualstats` (predicate counters), but *no autonomous create/drop loop* — a DBA still has to interpret recommendations and act. Our project closes that gap: a PostgreSQL extension, `auto_index`, that observes the live workload and creates and drops indexes (single-column or composite, up to three columns) entirely on its own.

2 Functionality implemented

The extension hooks into PostgreSQL in three places. An `ExecutorEnd` hook runs after every query, walks the plan tree, finds every `SeqScanState`, and decodes its filter quals into `(attno, op)` pairs. The decoder handles `Var op Const`, `IN (...)`, `BETWEEN`, `AND/OR` via `BoolExpr` (so even TPC-H Q19’s deeply nested OR-of-AND extracts correctly), and implicit casts via `RelabelType`. A fixed-size shared-memory area (`AutoIndexSharedState`) holds per-table predicate counters: per column “singleton” counts and a separate “colset” bucket that records when 2–3 columns co-occur in the same query. A background worker wakes every `check_interval` seconds, folds interval counters into cumulative state, runs the active create-strategy and the drop logic, and emits `CREATE INDEX / DROP INDEX` directly via SPI.

Composite indexes (up to 3 columns). When the same group of 2–3 columns repeatedly co-occurs, the worker builds a composite. Column order at create time puts equality columns first, range columns last (B-tree cannot use later columns past a range), with `attno` as the tiebreak. To avoid redundant work, the singleton index on the composite’s *leading* column is suppressed (subsumed by the composite’s leftmost prefix), but singletons on non-leading columns are kept because B-tree cannot use them on their own.

Pluggable create-strategies. Six strategies are selectable through the `auto_index.create_strategy` GUC:

Strategy	Rule
<code>ski_rental</code> (<i>default</i>)	$\text{cumulative_benefit} \geq \lceil \text{build_cost} / \text{seq_scan_cost} \rceil$, AND read/write ratio above threshold
<code>simple_threshold</code>	$\text{cumulative_benefit} \geq N$
<code>ratio_only</code>	read/write ratio above threshold
<code>cost_gain</code>	projected savings minus maintenance must pay off <code>build_cost</code>
<code>size_gated</code>	<code>simple_threshold</code> + minimum table size
<code>always</code>	fire on any positive benefit

Each strategy is a single `decide_create_*(ctx) → bool` function; the dispatcher is one switch.

Drop logic. Drop is ski-rental in reverse. For every auto-created index (we keep our own catalog table that survives restart) the worker reads `pg_stat_user_indexes.idx_scan` each interval. If `idx_scan` increased, the idle accumulator is reset; otherwise $(\text{writes} \times \text{maint_per_write} + \text{seq_scan_cost}) \times 1000$ is added. Once the accumulator passes `build\_cost` $\times 1000$ the index is dropped. The `seq\_scan\_cost` floor ensures even zero-write tables eventually age out an index that is never read.

SQL surface. `auto_index_reset()`, `auto_index_stats()`, and the catalog view `auto_index_catalog` (with the helper `auto_index_attnames(relid, attnos)` for human-readable column names).

Scope limitations (documented in code, not security-critical): predicate extraction only fires on `SeqScanState` — once any other index exists on a table, the planner switches to `Bitmap Heap Scan` and our hook can no longer see filter predicates (the dominant case in OLTP, capping TPC-C gains around 1.14 $\times$). Aggregate-bound queries (TPC-H Q01 doing `SUM` over the whole table) cannot be helped by any index in principle.

3 Code created

All new code lives under `contrib/auto_index/` (the extension) and `scripts/` (build and benchmark drivers). *No upstream PostgreSQL source files were modified* — the extension lives entirely in `contrib/`, hooks into existing extension points, and links cleanly against an unmodified server.

Path	Purpose	Li
<code>contrib/auto_index/auto_index.c</code>	core: hooks, shmem, bgworker, parsing, strategies, DDL emission	18
<code>contrib/auto_index/auto_index.h</code>	shared structs and GUCs declaration	68
<code>contrib/auto_index/auto_index-1.0.sql</code>	SQL functions and catalog table	37
<code>contrib/auto_index/auto_index.control, Makefile</code>	extension manifest, PGXS build	—
<code>scripts/{build,run,benchmark,demo}.sh</code>	top-level driver scripts	~2
<code>scripts/lib/*.sh</code>	per-benchmark helpers (pgbench, TPC-C, TPC-H, strategies, setup)	~7
<code>scripts/tpch_queries/q*.sql</code>	8 representative TPC-H queries	—
<code>scripts/tpcc_xn/*.sql, schema/load</code>	TPC-C-lite schema and the four transactions	—
<code>Dockerfile, docker-compose.yaml</code>	reproducible build container	—

The C source is split into six clearly-labelled sections inside `auto_index.c`: 1. `MODULE INIT & GUCs`, 2. `SHARED MEMORY`, 3. `PARSING`, 4. `STRATEGIES`, 5. `INDEX MANAGEMENT`, and 6. `SQL-CALLABLE FUNCTIONS`.

4 How we tested

Four benchmarks, scripted under `scripts/benchmark.sh`. All numbers come from a Docker container (Ubuntu 22.04, 6 cores, 8 GB RAM, NVMe SSD), PostgreSQL 19devel built with `-O2 -enable-debug -enable-cassert`. Each scenario was run three times.

pgbench live demo. 300 K-row table, 4 clients $\times$ 2 worker threads, single equality predicate; TPS reported every 2s.

Window	Run 1	Run 2	Run 3	Phase
$t = 2$ s	380	342	316	seq-scan baseline
$t = 4$ s	340	337	330	seq-scan baseline
$t = 6$ s	14 696	15 116	15 499	auto_index fires
$t = 8$ s	17 571	18 080	17 859	indexed (steady)
$t = 10\text{--}18$ s	17 200–18 400	17 650–18 460	17 500–18 400	indexed (steady)

The step happens at $t = 6$ s in every run — the bgworker’s second snapshot. Steady-state TPS is roughly 17 800 against a baseline of ~ 340 : a factor of ~ 52 in throughput, with no human asking for an index.

Query	Baseline (ms)	Speedup vs. baseline (higher is better)					
		ski_rental	simple_thr	ratio_only	cost_gain	size_gated	always
q01	4916	1.05×	1.04×	1.04×	1.02×	1.02×	0.99×
q03	978	1.07×	0.99×	1.02×	1.10×	0.98×	0.96×
q05	650	1.48×	0.94×	1.60×	1.64×	0.55×	1.05×
q06	601	1.57×	2.61×	3.20 ×	1.50×	1.65×	2.62×
q10	1016	1.28×	1.22×	1.24×	1.30×	1.12×	1.24×
q12	1011	1.47×	1.96×	2.06 ×	1.48×	1.75×	1.94×
q14	551	2.80×	2.52×	2.53×	2.91 ×	2.51×	2.49×
q19	855	2.13×	2.82×	3.07 ×	2.18×	2.78×	2.93×
geomean	—	1.53×	1.60×	1.79 ×	1.55×	1.32×	1.61×
indexes built	—	6	11	10	6	10	13

TPC-H, all six strategies. Same TPC-H workload (8 representative queries: Q1, Q3, Q5, Q6, Q10, Q12, Q14, Q19) under each of the six strategies. The same baseline phase was used as a reference for all strategies; for each strategy we restarted the server, ran 5 training passes, waited 20 s, and measured.

ratio_only comes out on top with a geomean of $\sim 1.79\times$, achieving $3\times+$ speedups on Q06, Q12, and Q19 by building the right composite indexes (notably the 3-column (`l_quantity`, `l_discount`, `l_shipdate`) that exactly matches Q06’s filter). **ski_rental**, the conservative default, still gets a respectable $1.53\times$ with only 6 indexes.

TPC-C-lite (OLTP). 2 warehouses, pgbench-driven mix of NewOrder / Payment / OrderStatus / StockLevel. The single auto-created index was on `orders.o_c_id`; **auto_index** correctly *avoided* indexing customer columns despite heavy reads, because Payment also writes to those rows. Mean speedup $1.14\times \pm 0.02\times$ across runs — a small but correct OLTP result, capped by the bitmap-heap-scan limitation noted above.

Lifecycle demo. 50 K-row table; phase A is 200 reads on a single column then 200 reads on a 3-column predicate; phase B is 300 INSERT/UPDATE/DELETE rounds with *no* reads on the indexed columns.

Run	Phase A: indexes built	Phase B: time to drop all	Result
1	4 (singletons + composite)	~ 20 s	✓
2	4 (singletons + composite)	~ 20 s	✓
3	4 (singletons + composite)	~ 20 s	✓

Every run produced both a singleton and a composite during phase A, and then dropped all of them roughly four bgworker cycles into phase B. The drop accumulator is deterministic enough that the ~ 20 s figure is practically a constant.

A late-stage docker-compose fix: PostgreSQL parallel hash joins back their shared hash tables with POSIX shm under `/dev/shm`, and Docker’s 64 MB default caused TPC-H Q10/Q12/Q14 to fail with “could not resize shared memory segment”. We set `shm_size: 1g` in `docker-compose.yaml` and added a named volume for `/usr/local/pgsql` so future compose-file edits don’t nuke the install.

5 How we used AI

AI was *not* used to design the high-level approach (the ski-rental analogy, the strategy-pluggable architecture, the choice of hook points); those came from team discussion. The line-by-line C, the qual-extraction code, the benchmark plumbing, and most of the internals research were AI-assisted.

We have read and verified every line of code that AI produced. Each chunk of AI-generated C went through compile + run + regression of an earlier benchmark before being committed; several AI-suggested approaches were rejected for being non-idiomatic (e.g. an early draft used `palloc` in shared memory; we rewrote with `ShmemAlloc`).

The kinds of prompts we used:

- Locating where filter quals live in the executor at runtime.
- The standard chain-and-restore idiom for an `ExecutorEnd_hook`.
- `shmem_request_hook` + `shmem_startup_hook` pattern.
- A recursive walker over `OpExpr` / `BoolExpr` / `ScalarArrayOpExpr` that peels `RelabelType`.

- Emitting `CREATE INDEX` from a bgworker safely (SPI).
- Which prefixes a multi-column B-tree can satisfy.
- An extension-owned catalog table that survives restart.
- Bash plumbing for the per-strategy comparison table.
- Debugging the Docker `/dev/shm` parallel-hash failure.
- Drafting prose from team-supplied bullet points and benchmark CSVs.

We have checked every line AI has written for correctness.

6 Submission zip layout

```

auto_index_submission/
|-- README.md                # build & run instructions
|-- git_url.txt              # link to public fork
|-- diffs/                   # per-file git diffs vs. upstream PostgreSQL
|   |-- .gitignore.diff      # the only modified upstream file
|-- new_files/               # all new files we authored, mirrored at original paths
|   |-- contrib/auto_index/...
|   |-- scripts/...
|   |-- Dockerfile, docker-compose.yaml
|-- test_data/               # test artefacts
|   |-- tpch_queries/q*.sql, tpch_schema.sql
|   |-- tpcc_schema.sql, tpcc_load.sql, tpcc_xn/*.sql
|-- report/
|   |-- submission_report.pdf # this document
|   |-- report.pdf            # full technical report (deeper dive)

```

The TPC-H raw data files (*.tbl, ~70 MB at SF=0.1) are not shipped — `scripts/lib/tpch_setup.sh` regenerates them with `tpch-dbgen` on first run.

Reproducing the headline results from a clean checkout.

```

git clone https://github.com/HypedUpb0ii/postgres.git
cd postgres
scripts/build.sh all          # docker + pg + init + extension
scripts/benchmark.sh pgbench # ~52x live step
scripts/benchmark.sh tpch    # 8-query 3-phase TPC-H
scripts/benchmark.sh tpcc    # OLTP
scripts/demo.sh              # full create-then-drop lifecycle, ~90 s

```